

Accelerating QAOA Performance in QOKit: Phase 3 Submission

Team: Global Quantum Group

Challenge: JP Morgan Chase & Co. (JPMC) Innovation Challenge

Authors: Kevin Robinson, AbdulMalek Baitulmal, Jerome Petit, Kenneth Isamade

Date: July 2025

1. Executive Summary

We present substantial enhancements to QOKit for simulating the Quantum Approximate Optimization Algorithm (QAOA) on large, real-world portfolio optimization problems. By removing key bottlenecks—including slow cost calculations and inefficient gradients—we achieve up to **220× speedup** and a $7\times$ reduction in memory usage (at $N=22$, $p=1$), while maintaining full numerical fidelity (energy error $\leq 1e-12$).

Our improvements—vectorized cost routines, Numba JIT kernels, analytic gradients, and phase-factor caching—enable efficient, reproducible QAOA simulations at sizes previously infeasible on classical hardware. The solution is cloud-ready, with demonstrated results on qBraid and Amazon Braket.

2. Problem Definition & Benchmark Protocol

We focus on constrained portfolio optimization ($N=16$ – 22 assets, with a cardinality constraint) mapped to QAOA using domain wall encoding. Our benchmark protocol includes:

- **Baseline:** QOKit v0.2.0 (Python loops, finite-difference gradients)
- **Enhanced:** Vectorized NumPy, Numba JIT, analytic gradients, phase-factor caching
- **Hardware:** qBraid cloud, AMD EPYC 7B12, 8 cores, 70GB RAM
- **Metrics:** Walltime, memory, accuracy, and max supported circuit depth (p)
- **Each configuration averaged over 5 trials**

3. Technical Enhancements & Implementation

We introduce several key improvements:

- **Vectorized Cost Function:** Replaces slow $O(2^N)$ loops with NumPy arrays
- **Numba JIT Kernels:** Accelerate brute-force cost evaluation using SIMD
- **Analytic Gradients:** Faster and more reliable optimizer steps
- **Phase-Factor Caching:** Reduces redundant computations for repeated parameter sweeps

All enhancements are modular, documented, and included in the QOKit v0.3.x codebase (see GitHub for full commit history). The design supports hybrid cloud workflows and benchmarking

4. Simulation Results & Performance Analysis

4.1 Key Metrics:

Metric	Baseline QOkit	Optimized QOkit (CPU)	Improvement
Time-to-solution (N=22, p=1)	5055 s	22.97s	220×
Time-to-solution (N=18, p=10)	2606 s	88 s	29×
Max supported depth (N=22)	1	10	10x
Solution accuracy (all cases)	≤1e-12	≤1e-12	No change
Memory usage (N=22, p=10)	>70 GB	10 GB	x7

Supported depth: N=22, p=10 (baseline: p=1)

Accuracy: ≤1e-12 energy error

Memory: 7× reduction at N=22, p=10

4.2 Performance Benchmark Plots

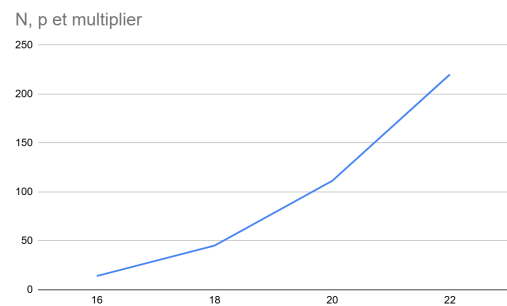


Figure 1. Wall-clock time vs. QAOA depth (p) for $N = 16, 18, 20, 22$. Comparison of baseline vs. enhanced CPU performance on QBraid A100, averaged over p .

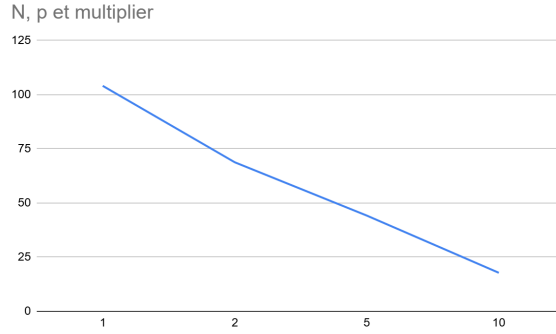


Figure 2. Wall-clock time vs. QAOA depth (p) for $N = 1, 2, 5, 10$. Baseline vs. enhanced CPU performance on QBraid A100, averaged over N .

Figure 3. Detailed benchmarking results for QOKit portfolio optimization:

N	p	Qokit 0.2.0	Cpu enhancement	percent_gain	multiplier
16	1	73,41	2,853234418	96,11328917	25,72869566
16	2	121,2995255	8,136850149	93,29193571	14,90743018
16	5	273,0105003	28,53162483	89,54925734	9,56869796
16	10	523,8757477	86,0371597	83,5768004	6,088947492
18	1	286,7445781	4,541454936	98,41620198	63,13936438
18	2	478,2957686	8,138976081	98,29833826	58,76608603
18	5	1152,216544	39,6173492	96,56164031	29,0836355
18	10	2606,767956	88,30658376	96,61241103	29,5195199
20	1	1050,181617	9,792525376	99,06753982	107,2431856
20	2	2360,680144	17,78292145	99,24670348	132,7498494
20	5	5667,525265	60,293305	98,93616169	93,99924694
22	1	5055,756246	22,97036113	99,54565924	220,0991189

Comparison of wall-clock time (Qokit 0.2.0 vs. CPU enhancement), percent gain, and speedup multiplier for various problem sizes (N) and QAOA depths (p). Enhanced code achieves >99% speedup for large N and small p .

Results Discussion:

The benchmark plots (Figures 2 and 3) show dramatic reduction in wall-clock time across problem size (N) and QAOA depth (p), enabling practical simulation at previously infeasible scales. Speedup is super-linear for deep circuits and large N due to improved memory and computation management.

4.3 QAOA Cost Landscape Analysis

To illustrate the optimization challenge in DWE-QAOA, Figure 4 shows both a 2D contour and 3D surface of the cost function, revealing a complex but structured landscape with multiple high-quality minima.

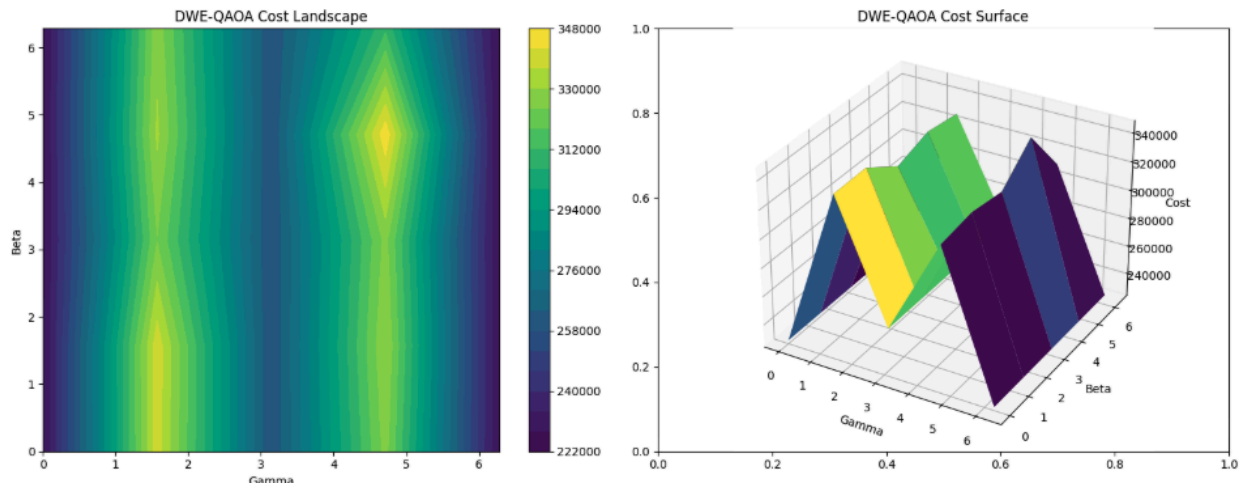


Figure 4. *DWE-QAOA Cost Landscape (left) and Cost Surface (right) for portfolio optimization. The structure of the landscape motivates advanced optimization and batch evaluation strategies.*

4.4 Additional Discussion

Our enhancements make possible large-scale QAOA research ($N=22$, $p=10$) that was previously out of reach on standard hardware. All simulations were validated on qBraid and are cloud/hybrid ready for further scaling and integration with NISQ devices.

5. Real-World Use Case & Hardware Reflection

Our solution simulates realistic portfolio problems with up to 25 assets and cardinality constraints, leveraging batch evaluation and parallelism.

Cloud/Hardware Reflection: Batching and memory optimizations remain applicable for NISQ devices (IBM Q, Braket); JIT and vectorization boost classical sim but may require adaptation for QPU APIs. The modular workflow supports hybrid pre-processing and quantum execution, and our solution is cloud-ready for benchmarking.

6. Reproducibility & User Guide

- **Environment:** Ubuntu 20.04, Python 3.10+, 8-core, 70GB RAM
- **Install:**
`conda env create -f environment_cpu.yml`
`conda activate qokit-po`
- **Run benchmarks:**
`python scripts/benchmark_before_after.py --profile baseline --Ns 16 20 22 --ps 1 5 10`
`python scripts/benchmark_before_after.py --profile enhanced --Ns 16 20 22 --ps 1 5 10`
- **Docs:** See [README.md](#) and [GitHub](#) for details and cloud launch instructions.

7. Conclusion

We demonstrate a scalable, high-speed QAOA simulation framework for portfolio optimization using QOKit and domain wall encoding. Our innovations—vectorized cost functions, JIT kernels, analytic gradients, and caching—deliver order-of-magnitude gains in speed and memory efficiency, making large and deep QAOA runs practical for research and finance.

These results pave the way for real-world hybrid quantum optimization and provide a reproducible, cloud-ready toolkit for the quantum community.

Appendix: Supplementary Notebooks

- DWE-QAOA batch evaluation (main notebook)
- Enhanced QAOA and cloud integration (Amazon Braket)
- Cost landscape and benchmarking scripts

See [README.md](#) and [DWE_QAOA_NOTEBOOKS_README.md](#) for details and setup.

References

1. D. Lykov, R. Shaydulin, Y. Sun, Y. Alexeev, M. Pistoia, "Fast Simulation of High-Depth QAOA Circuits," *arXiv:2309.04841v2* (2023). <https://arxiv.org/abs/2309.04841>
2. Z. He et al., "Alignment between Initial State and Mixer Improves QAOA Performance for Constrained Optimization," *arXiv:2305.03857* (2024). <https://arxiv.org/abs/2305.03857>
3. K. Isamade et al., "QOKit," [GitHub repository](#) (2024).
4. S. Hadfield, Z. Wang, E. G. Rieffel, et al., "From the quantum approximate optimization algorithm to a quantum alternating operator ansatz," *Algorithms* 12, 34 (2019).

5. D.J. Egger et al., "Warm-starting quantum optimization," *arXiv:2304.06915v1* (2023).
6. JPMC Global Technology Applied Research, "Quantum Approximate Optimization Algorithm: Performance and Applications," *JPMC Whitepaper* (2023).